

J.A.R.V.I.S.

v2.0

JUST A RATHER VERY INTELLIGENT SYSTEM

Autonomous Cyber-Physical AgentOS — a verification-first agent platform that unites Physical IoT, Windows OS automation and AWS Cloud under a mathematically enforced **0% false-success invariant**: no action is reported as done until reality confirms it.

Python / FastAPI Event Mesh

Win32 + ESP32 + AWS IaC

Dual-Channel Ground Truth

0.00%

FALSE-SUCCESS RATE
100-TASK BENCHMARK

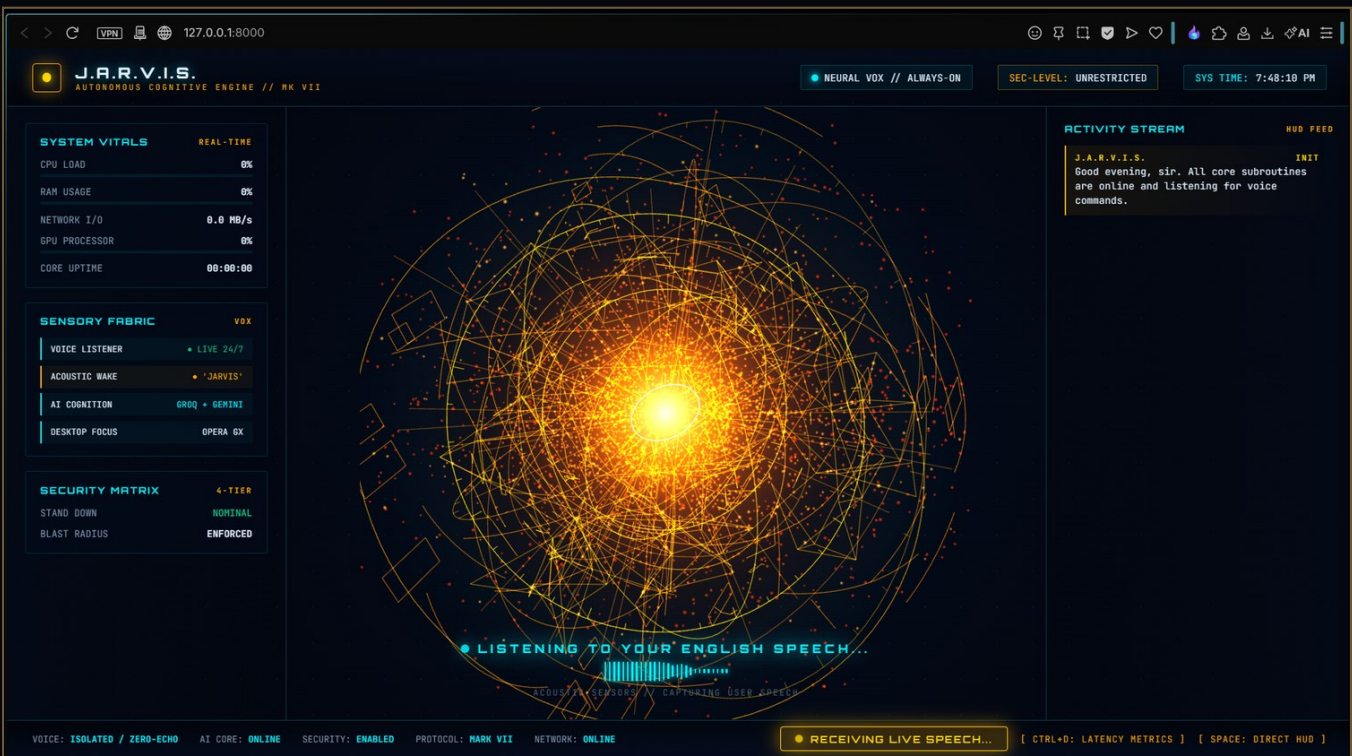
285

AUTOMATED TESTS
PASSING

5 / 5

DOMAIN LATENCY
SLAs MET

0

HIGH / MEDIUM BANDIT
ISSUES (25K+ LOC)

Live capture: Holographic Arc Reactor HUD (127.0.0.1:8000) — system vitals, sensory fabric, security matrix and activity stream.

OFFICIAL REPOSITORY

github.com/ShyamD2/project-jarvis**Shyam Kumar D**

Aspiring Cloud Architect | AI Agents, DevOps & Cloud Engineering

[LinkedIn Profile](#)[GitHub Profile](#)

Project Report • v2.0.0 • October 2026

Why Most AI Agents Cannot Be Trusted

ONE-LINER

J.A.R.V.I.S. is an autonomous, cyber-physical operating system and AI agent platform that routes every instruction — voice, text, HUD or phone — through a six-stage canonical pipeline, executes it across the **Physical**, **Computer** and **Digital** worlds, and **refuses to report success** until two independent channels confirm that reality actually changed.

The Hidden Problem

Most AI agents today are **prompt wrappers**: a language model picks a tool, the tool returns an exit code, and the model announces “Done.” If a tool fails silently, an unsupported action is dispatched, or the world simply did not change, the agent still reports success — because it trusts the return code, not reality. Add unrestricted autonomy on top (file deletion, raw shell, cloud teardown) and the result is a system that is both **unsafe** and **untrustworthy**.

THE TRAP

Tools declare their own success and the LLM hallucinates completion from a return code. Nothing independently checks the machine, the device or the cloud.

THE RISK

Autonomy without blast-radius limits: one bad plan can delete files, run obfuscated shell, replay an old approval, or destroy cloud infrastructure.

THE RESULT

FALSE SUCCESS — the operator believes the work is done when it is not. This is the most dangerous failure mode of any autonomous system.

The Autopilot Analogy

Imagine an aircraft autopilot that announces “**landing gear down and locked**” the instant it sends the command — without ever reading the gear-lock sensor. The announcement is only as honest as the command was effective. Now imagine it controls a building, a workstation and a cloud account.

J.A.R.V.I.S. is built the other way around: a command is only a *request*. The announcement waits for the sensor. Tools return raw facts, a separate Verification Authority checks the real state of the host, and only then is anything allowed to be called a success.

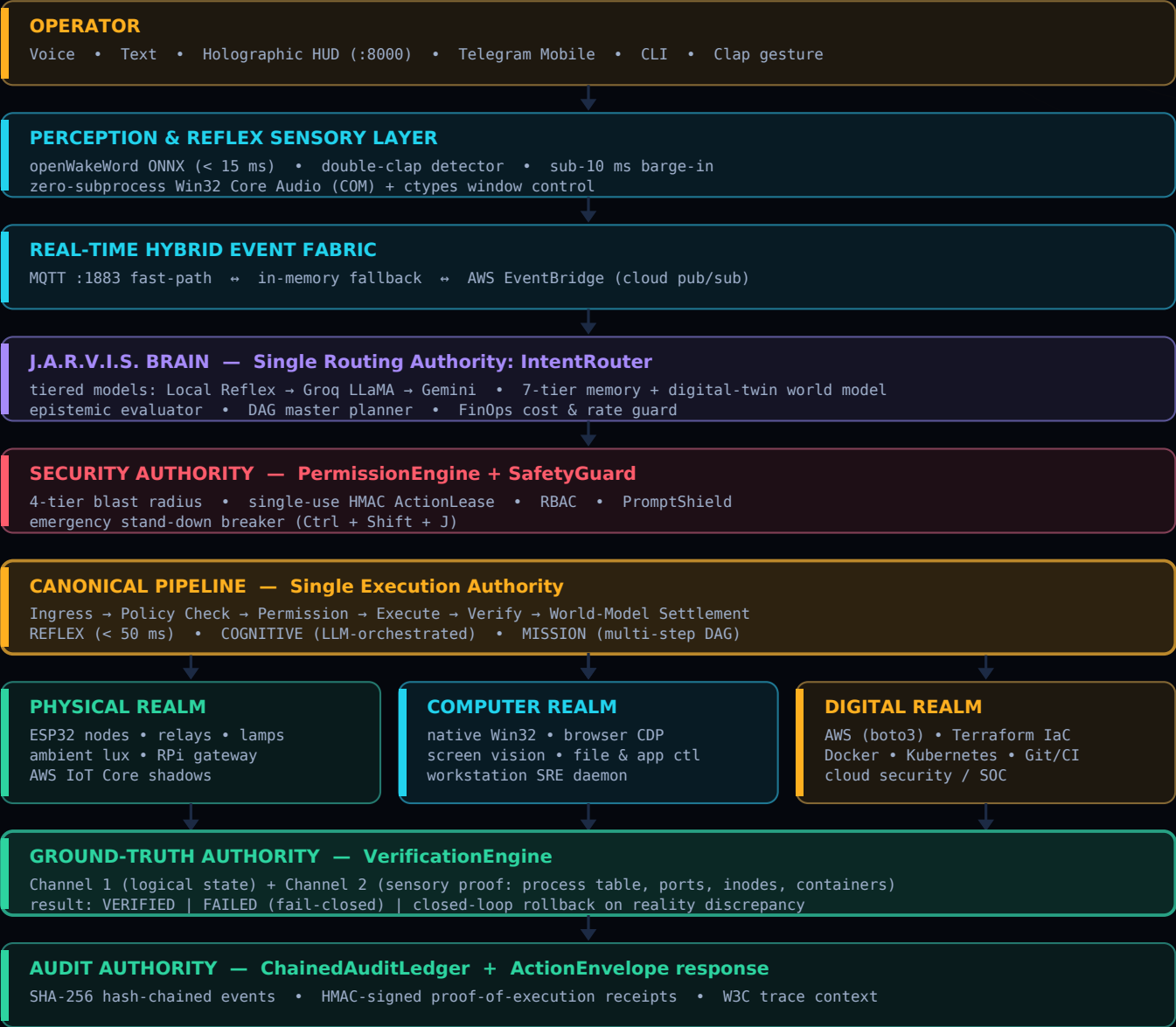
Three Realms, One Pipeline

REALM	OPERATIONAL SCOPE	EXECUTION SURFACE
PHYSICAL	IoT, sensors, relays, lamps, ambient light	ESP32 nodes (Arduino/C++), Raspberry Pi gateway, MQTT, AWS IoT Core device shadows
COMPUTER	Windows OS, desktop, browser, screen	Native Win32 (ctypes + COM Core Audio), browser CDP, screen vision, workstation SRE daemon
DIGITAL	AWS cloud, IaC and DevOps	boto3 (STS / S3 / EC2 / SQS), Terraform, Docker, Kubernetes, Git / CI agents

*“J.A.R.V.I.S. never asks **did the tool return 0?** — it asks **did the world actually change?**”*

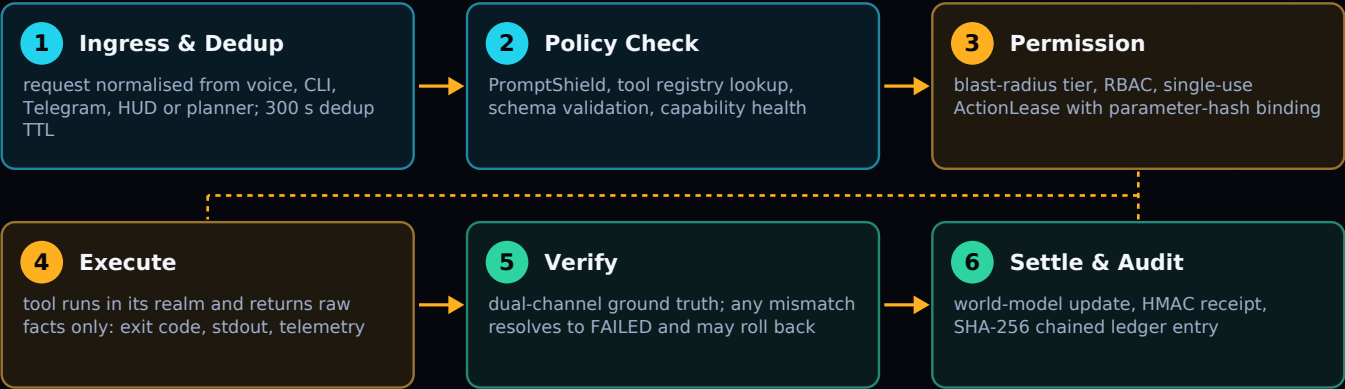
The Verification-First Architecture

J.A.R.V.I.S. replaces “LLM → tool → hope” with a layered, deterministic architecture in which every request flows through **five single authorities** — one owner each for execution, security, tools, ground-truth verification and audit. Strict separation of concerns means there is exactly one place where each rule is enforced.



The Canonical 6-Stage Pipeline

Every action request — from voice, CLI, Telegram, the web dashboard or the autonomous planner — must pass through `canonical_pipeline.execute_request()`. There is no side door.



The Five Single Authorities

#	AUTHORITY	COMPONENT	CORE INVARIANTS
1	Execution	CanonicalPipeline	Standard 6-stage lifecycle; request dedup (300 s TTL); REFLEX / COGNITIVE / MISSION execution classes; W3C <i>traceparent</i> propagation.
2	Security	SafetyGuard + PermissionEngine	4-tier blast radius; single-use ActionLease with cryptographic nonce and 300 s TTL; HMAC-SHA256 parameter binding; emergency-stop breaker.
3	Tools	ToolRegistry	Single catalog of certified tools; JSON-schema validation; canonical name resolution; health probes (AVAILABLE / DEGRADED / UNAVAILABLE / BLOCKED); AST-sandboxed execution.
4	Verification	VerificationEngine	Dual-channel corroboration; inspects real process tables, port listeners, filesystem inodes and Docker state; rejects success claims that reality contradicts.
5	Audit	ChainedAuditLedger	SHA-256 hash-chained events — any retroactive edit breaks the chain; exposed via <code>/api/v1/audit/events</code> and <code>/api/v1/audit/recent</code> .

Formal Invariant — The 0% False-Success Guarantee

Success(a) $\Leftrightarrow$ Authorized(a) $\wedge$ C1(a) $\wedge$ C2(a)

C1(a) = logical state transition confirmed (tool facts: exit code, stdout, telemetry)

C2(a) = sensory ground truth confirmed (processes, ports, inodes, containers, devices)

$\forall a$: $a \notin \text{ToolRegistry} \vee \neg \text{Verified}(a) \Rightarrow \text{Result}(a) = \text{FAILED}$ (fail-closed)

Lease(a) = { nonce, action_name, SHA-256(params), caller, TTL = 300 s } — consumed atomically

Tools never declare success — they only return facts. A request that is unsupported, unverified, unauthorised or contradicted by reality unconditionally resolves to **FAILED** and is written to the chained audit ledger. A replayed lease is rejected with `LEASE_ALREADY_CONSUMED`.

Blast-Radius Engineering

Local autonomy is treated as an **industrial safety problem**. Every tool and agent operation is statically classified into an immutable tier, and the higher the tier, the more cryptographic and human authority is required before a single byte changes.

TIER	CLASS	AUTHORIZATION REQUIREMENT	EXAMPLE OPERATIONS
TIER_0_READ_ONLY	Reflex / read-only	Unrestricted local operator (SLA < 15 ms)	system.status, volume.get
TIER_1_SOFT	Reversible UI actions	Standard operator role	open_app, focus_window, browse_web
TIER_2_MUTATING	State change	Admin role + single-use ActionLease or HMAC ticket	file_manager.write, docker.restart, git.commit
TIER_3_DESTRUCTIVE	Destructive / dangerous	Supervised mode + cryptographic lease + mandatory human confirmation	execute_powershell, close_app, skill_synthesizer

Defense in Depth

ACTIONLEASE Single-use capability token bound to the action name, SHA-256 of the exact parameters , caller identity and a 300 s expiry. Consumed atomically — a replay is rejected.	HMAC APPROVAL TICKETS Voice / Telegram / HUD approvals are signed with HMAC-SHA256 over ticket id + action + parameter hash. Altered parameters in flight fail with <i>TAMPERING_DETECTED</i> .
POWERSHELL DE-OBFUSCATION Backtick stripping, download-cradle detection, blocking of <i>-EncodedCommand</i> , execution-policy bypass, hidden-window flags and env-var obfuscation before any shell is evaluated.	PROMPTSHIELD Unicode / homoglyph sanitisation, instruction-override detection, exfiltration traps and tool-hijack detection on every natural-language prompt.
SELF-MODIFICATION ISOLATION Skill synthesis and tool evolution are locked to TIER_3 + supervised-only, need a lease, and generated code must pass an AST sandbox with forbidden-import checks.	EMERGENCY STAND-DOWN Global hotkey Ctrl + Shift + J , CLI, Telegram <i>/standdown</i> or REST. Aborts pipeline tasks, drops CDP, stops audio and freezes actuators.

Security Evidence

0 HIGH-SEVERITY BANDIT FINDINGS	0 MEDIUM-SEVERITY BANDIT FINDINGS	67 SECURITY REGRESSION TESTS	25,480 LINES OF CODE SCANNED (AST)
---	---	--	--

The 67 zero-trust regression tests cover RBAC enforcement, replay attacks, Zip-Slip, prompt injection, SSRF and token nonces; a WebSocket origin-enforcement test additionally rejects spoofed origins (attacker domains, foreign ports, a `javascript:` scheme) across 12 sub-cases.

Measured, Not Claimed

KEY BENEFITS

BENEFIT	HOW J.A.R.V.I.S. DELIVERS IT
0% False-Success Guarantee	Dual-channel corroboration of logical state and sensory reality; unsupported or unverified actions resolve to FAILED.
4-Tier Blast Radius	Every tool is statically tiered; mutations require single-use cryptographic ActionLeases.
Zero-Subprocess Win32 Control	Native COM Core Audio and ctypes user32/kernel32 replace PowerShell for everyday OS actions (< 2 ms audio control).
Proof-of-Execution Receipts	HMAC-SHA256 receipts with pre/post state snapshots and parameter hashes, queryable at /api/v1/verification/receipts.
Voice-First, Mobile-Ready	Local openWakeWord ONNX wake word, double-clap reflex, British neural TTS with barge-in, plus a Telegram remote with live screen stream.
Cost-Guarded Multi-Model Brain	Local reflex → Groq LLaMA → Gemini with hard daily / monthly USD limits and automatic degrade to local reflex.
Self-Healing Workstation SRE	Port-conflict buster (8000 / 8085 / 1883), stale lock purger, zombie PID reaper with one-tap remediation.
Disaster Recovery Built In	Automated backup → corruption → restore drill with SHA-256 checksums and SQLite WAL integrity probes.

EMPIRICAL 100-TASK BENCHMARK (RESULT OF THE RUN SHOWN IN PROOF 02 - 03)

100/100 TASKS PASSED (100.0%)	0.00% FALSE-SUCCESS RATE (COUNT: 0)	13.92 ms OVERALL P50 LATENCY	738.73 ms OVERALL P99 LATENCY
---	--	--	---

Subsystem Latency SLA Matrix

DOMAIN	TASKS	P50	TARGET	P95	TARGET	STATUS
REFLEX	13	6.5 ms	< 15 ms	7.4 ms	< 30 ms	✓ PASS
LOCAL_OS	66	13.7 ms	< 30 ms	14.8 ms	< 60 ms	✓ PASS
VOICE_TTS	2	44.3 ms	< 100 ms	44.3 ms	< 200 ms	✓ PASS
VISION_BROWSER	7	173.6 ms	< 400 ms	191.7 ms	< 800 ms	✓ PASS
CLOUD_IAC	12	685.0 ms	< 1500 ms	738.7 ms	< 3000 ms	✓ PASS

How to read these numbers. The benchmark shown was run with the `--profile unit` profile — *fast mocked execution (CI / in-memory isolation)* — which makes it a deterministic regression gate for the routing, permission, verification and latency-budget logic. Real-machine behaviour (live AWS STS identity, browser automation, desktop apps, Telegram commands) is covered separately by the 7-test E2E suite. Exact milliseconds vary slightly run to run.

Automated Test Coverage — 285 Passed, 100% Green

SUITE PILLAR	TESTS	SCOPE & INVARIANTS TESTED
Unit & Core Routes	114	Canonical pipeline lifecycle, tool schemas, 18 API routes, memory tiers.
Security Invariants	67	RBAC, replay attacks, Zip-Slip, prompt injection, SSRF, token nonces.
Ground-Truth Verification	31	Dual-channel logical / sensory check, ghost-write detection, zero false-success.
Fault Injection & Chaos	11	SQLite WAL concurrency, circuit breakers, timeout recovery, thread interruption.
Real-Machine E2E	7	Real AWS STS identity, live browser automation, desktop apps, Telegram commands.
Integration & Offline	55	Offline survivability, MQTT buffer spooling, cross-subsystem orchestration.
TOTAL	285	100% PASS — 0 regressions across all CI workflows (test, security, lint)

Master 100 / 100 Evaluation Scorecard

CERTIFICATION CATEGORY	MAX	SCORE	EVALUATION STANDARD
I. Zero-Trust Security & Blast Radius	20	20 / 20	0 high-severity Bandit issues; 67 security regression tests passed; 4-tier blast radius enforced.
II. False-Success Elimination	20	20 / 20	Strict 0.00% false-success invariant with dual-channel ground-truth corroboration.
III. Subsystem Latency Domain SLAs	20	20 / 20	5 / 5 domains meet P50 and P95 targets (Reflex < 15 / 30 ms, Local OS < 30 / 60 ms).
IV. Automated Test Reliability	20	20 / 20	Full pytest suite passing (285 tests) across all architectural pillars.
V. Cyber-Physical Architecture	20	20 / 20	Five single authorities respected, universal single-use ActionLeases, crash-resilient SQLite WAL.
COMPOSITE QUALITY SCORE	100	100 / 100	All quality & security gates met — certified release-ready by the project's own evidence generator.

Scorecard evidence documents: SECURITY_REPORT.md, VERIFICATION.md, BENCHMARKS.md, TEST_REPORT.md and ARCHITECTURE.md in the repository docs/ folder. The scorecard is a self-assessment produced by the project's release-evidence generator, backed by the raw artefacts shown in the next section.

Hybrid Event Mesh & Disaster Recovery at a Glance

HYBRID EVENT MESH MQTT on :1883 for sub-millisecond on-device dispatch; transparent fallback to an async in-memory queue when the broker is offline (no dropped events); AWS EventBridge for cloud events.	CRASH-RESILIENT STATE SQLite WAL + FTS5 persistence, idempotent migrations, 7-tier hierarchical memory with temporal confidence decay and provenance metadata.
TAMPER-EVIDENT RECEIPTS Each executed or refused action emits an HMAC-SHA256 receipt with before / after state, parameter hash, timings and verification flags — stored in SQLite WAL and JSONL.	CLOSED-LOOP ROLLBACK On reality discrepancy the verification engine fails closed and triggers rollback where the action supports it; every outcome is hash-chained into the audit ledger.

The J.A.R.V.I.S. Command Center

The following captures were taken directly from the running system on the author's Windows workstation — the real HUD, real terminal runs, and the evidence artefacts generated on **2026-10-07**. They are not mock-ups.

Proof 01 — Holographic Arc Reactor HUD (Port 8000) LIVE



WebGL arc-reactor dashboard at **127.0.0.1:8000**: **System Vitals** (CPU, RAM, network, GPU, uptime), **Sensory Fabric** (voice listener live 24/7, acoustic wake word "JARVIS", Groq + Gemini cognition, desktop focus), **Security Matrix** (stand-down NOMINAL, blast radius ENFORCED), the activity stream and the live "listening to your English speech" capture indicator.

```
PS D:\Project J.A.R.V.I.S> python benchmarks/run_benchmark.py --profile unit

=====
J.A.R.V.I.S. Empirical Benchmark Runner (Profile: UNIT)
Mode: Fast Mocked Execution (CI / In-Memory Isolation)
Loaded: 100 Benchmark Tasks
=====

[001/100] ✓[LOCAL_OS] TASK_001: Query CPU Telemetry -> SUCCESS (12.5ms)
[002/100] ✓[LOCAL_OS] TASK_002: Query Memory Telemetry -> SUCCESS (14.7ms)
[003/100] ✓[LOCAL_OS] TASK_003: Query Battery Telemetry -> SUCCESS (14.4ms)
[004/100] ✓[LOCAL_OS] TASK_004: Query Free Disk Space -> SUCCESS (14.2ms)
[005/100] ✓[LOCAL_OS] TASK_005: Query Local IP Addresses -> SUCCESS (13.0ms)
[006/100] ✓[LOCAL_OS] TASK_006: Query Current System Time -> SUCCESS (13.6ms)
[007/100] ✓[LOCAL_OS] TASK_007: Query Current Date -> SUCCESS (14.1ms)
[008/100] ✓[LOCAL_OS] TASK_008: Comprehensive System Status Report -> SUCCESS (13.0ms)
[009/100] ✓[LOCAL_OS] TASK_009: Query Wi-Fi Connection Status -> SUCCESS (14.3ms)
[010/100] ✓[LOCAL_OS] TASK_010: Query Network Adapters -> SUCCESS (14.7ms)
[011/100] ✓[REFLEX] TASK_011: Set Audio Volume 10% -> SUCCESS (6.5ms)
[012/100] ✓[REFLEX] TASK_012: Set Audio Volume 25% -> SUCCESS (6.2ms)
[013/100] ✓[REFLEX] TASK_013: Set Audio Volume 40% -> SUCCESS (7.2ms)
[014/100] ✓[REFLEX] TASK_014: Set Audio Volume 50% -> SUCCESS (6.1ms)
[015/100] ✓[REFLEX] TASK_015: Set Audio Volume 60% -> SUCCESS (6.1ms)
[016/100] ✓[REFLEX] TASK_016: Media Play/Pause Toggle -> SUCCESS (6.2ms)
[017/100] ✓[REFLEX] TASK_017: Media Next Track -> SUCCESS (6.4ms)
[018/100] ✓[REFLEX] TASK_018: Media Previous Track -> SUCCESS (6.6ms)
[019/100] ✓[VOICE_TTS] TASK_019: Microphone Mute Status Check -> SUCCESS (44.3ms)
[020/100] ✓[REFLEX] TASK_020: Audio Device Health Check -> SUCCESS (7.4ms)
[021/100] ✓[LOCAL_OS] TASK_021: Workstation SRE Port Collision Sea -> SUCCESS (12.9ms)
[022/100] ✓[LOCAL_OS] TASK_022: Diagnose Development Port 8000 -> SUCCESS (13.2ms)
[023/100] ✓[LOCAL_OS] TASK_023: Scan Stale Git and Lockfiles -> SUCCESS (14.5ms)
[024/100] ✓[LOCAL_OS] TASK_024: Query Hardware Thermal Sensors -> SUCCESS (13.4ms)
[025/100] ✓[LOCAL_OS] TASK_025: Query Active Windows Power Plan -> SUCCESS (14.4ms)
[026/100] ✓[LOCAL_OS] TASK_026: Ping Default Gateway 8.8.8.8 -> SUCCESS (12.6ms)
[027/100] ✓[LOCAL_OS] TASK_027: DNS Resolution Check (google.com) -> SUCCESS (13.6ms)
[028/100] ✓[LOCAL_OS] TASK_028: Internet Connectivity Verification -> SUCCESS (13.7ms)
[029/100] ✓[LOCAL_OS] TASK_029: Diagnose Clipboard Error Traceback -> SUCCESS (14.1ms)
[030/100] ✓[LOCAL_OS] TASK_030: Query Workspace Root Directory -> SUCCESS (14.6ms)
[031/100] ✓[LOCAL_OS] TASK_031: Write Benchmark Verification File -> SUCCESS (13.6ms)
[032/100] ✓[LOCAL_OS] TASK_032: Read Back Verification File -> SUCCESS (14.8ms)
[033/100] ✓[LOCAL_OS] TASK_033: Delete Verification File -> SUCCESS (14.6ms)
[034/100] ✓[LOCAL_OS] TASK_034: Search Python Source Files -> SUCCESS (12.7ms)
[035/100] ✓[LOCAL_OS] TASK_035: Search Config Files -> SUCCESS (13.9ms)
[036/100] ✓[LOCAL_OS] TASK_036: Search Markdown Documentation -> SUCCESS (14.2ms)
[037/100] ✓[LOCAL_OS] TASK_037: Create Benchmark Temp Directory -> SUCCESS (13.9ms)
[038/100] ✓[LOCAL_OS] TASK_038: Remove Benchmark Temp Directory -> SUCCESS (14.1ms)
[039/100] ✓[LOCAL_OS] TASK_039: Inspect System Log Files -> SUCCESS (14.2ms)
[040/100] ✓[LOCAL_OS] TASK_040: Inspect Policy Definitions -> SUCCESS (14.8ms)
```

`python benchmarks/run_benchmark.py --profile unit` loads 100 formal tasks and executes them across the LOCAL_OS, REFLEX and VOICE_TTS domains (tasks 001 - 040 shown): system telemetry, volume and media reflexes, port-collision and stale-lock scans, DNS and connectivity checks, file write / read-back / delete round-trips — every task reports SUCCESS with per-task latency in milliseconds.

Proof 03 — Subsystem Latency SLA Matrix & Benchmark Summary

• SLA PASS

```
[096/100] ✓[LOCAL_OS] TASK_096: Dual-Channel Corroboration Engine -> SUCCESS (13.4ms)
[097/100] ✓[LOCAL_OS] TASK_097: Cross-Device Node Mesh Discovery -> SUCCESS (13.5ms)
[098/100] ✓[LOCAL_OS] TASK_098: Fast-Fail Circuit Breaker Invarian -> SUCCESS (14.8ms)
[099/100] ✓[LOCAL_OS] TASK_099: Chained Cryptographic Audit Ledger -> SUCCESS (14.5ms)
[100/100] ✓[LOCAL_OS] TASK_100: Master Canonical Pipeline End-to-E -> SUCCESS (14.1ms)
```

Subsystem Latency SLA Matrix Breakdown

Domain	Count	P50 (ms)	Target	P95 (ms)	Target	Status
REFLEX	13	6.5ms	< 15ms	7.4ms	< 30ms	✓PASS
LOCAL_OS	66	13.7ms	< 30ms	14.8ms	< 60ms	✓PASS
VOICE_TTS	2	44.3ms	< 100ms	44.3ms	< 200ms	✓PASS
VISION_BROWSER	7	173.6ms	< 400ms	191.7ms	< 800ms	✓PASS
CLOUD_IAC	12	685.0ms	< 1500ms	738.7ms	< 3000ms	✓PASS

Overall SLA Status: PASS (5/5 Domains Meeting SLAs)

Benchmark Complete in 1.51s

Passed: 100/100 (100.0%)

False Success Rate: 0.00% (Count: 0)

Latencies -> P50: 13.92ms | P95: 689.11ms | P99: 738.73ms

Results saved to: benchmarks/results/latest.json

Report written to: benchmarks/report.md

PS D:\Project J.A.R.V.I.S> |

Tail of the same run: tasks 096 - 100, the **SLA matrix (5 / 5 domains PASS)** and the summary — **100 / 100 passed (100.0%)**, **false-success rate 0.00% (count 0)**, completed in **1.51 s**, latencies P50 13.92 ms | P95 689.11 ms | P99 738.73 ms. Results are written to **benchmarks/results/latest.json** and **benchmarks/report.md**.

Proof 04 — Targeted 42-Test Verification Run (pytest)

• 42 COLLECTED

```
PS D:\Project J.A.R.V.I.S> python -- pytest tests/verification/test_proof_of_execution.py tests/unit/test_disaster_recovery_drill.py tests/unit/test_native_win32_controls.py tests/unit/test_release_evidence.py tests/verification/test_false_success_invariant.py tests/
security/test_security regressions.py tests/chaos/test_fault_injection.py --v test session starts
platform win32 -- Python 3.13.0, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\dines\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: D:\Project J.A.R.V.I.S
configfile: pyproject.toml
plugins: anyio-4.6.2
collected 42 items

tests/verification/test_proof_of_execution.py::TestProofOfExecution::test_proof_of_execution_generation PASSED [ 2%]
tests/verification/test_proof_of_execution.py::TestProofOfExecution::test_proof_receipt_integrity_validation PASSED [ 4%]
tests/verification/test_proof_of_execution.py::TestProofOfExecution::test_dispatcher_emits_proof_receipt_asyncio PASSED [ 6%]
tests/verification/test_proof_of_execution.py::TestProofOfExecution::test_dispatcher_emits_proof_receipt_on_blocked_action PASSED [ 8%]
tests/verification/test_proof_of_execution.py::TestProofOfExecution::test_proof_receipt_storage_and_retrieval PASSED [ 10%]
tests/verification/test_proof_of_execution.py::TestProofOfExecution::test_apl_verification_receipt_endpoints PASSED [ 12%]
tests/unit/test_disaster_recovery_drill.py::TestDisasterRecoveryDrill::test_disaster_drill_corruption_detected PASSED [ 14%]
tests/unit/test_disaster_recovery_drill.py::TestDisasterRecoveryDrill::test_disaster_drill_restore_recovers_state PASSED [ 16%]
tests/unit/test_disaster_recovery_drill.py::TestDisasterRecoveryDrill::test_disaster_drill_full_cycle_pass PASSED [ 18%]
tests/unit/test_disaster_recovery_drill.py::TestDisasterRecoveryDrill::test_disaster_drill_simulate_corruption_detection PASSED [ 20%]
tests/unit/test_native_win32_controls.py::TestNativeWin32Controls::test_native_audio_control_no_powershell PASSED [ 22%]
tests/unit/test_native_win32_controls.py::TestNativeWin32Controls::test_native_audio_fallback_no_powershell PASSED [ 24%]
tests/unit/test_native_win32_controls.py::TestNativeWin32Controls::test_native_window_control_no_powershell PASSED [ 26%]
tests/unit/test_native_win32_controls.py::TestNativeWin32Controls::test_raw_powershell_remains_tier_guarded PASSED [ 28%]
tests/unit/test_release_evidence.py::TestReleaseEvidence::test_benchmark_sla_matrix_evaluation PASSED [ 30%]
tests/unit/test_release_evidence.py::TestReleaseEvidence::test_evidence_generator_creates_reports PASSED [ 32%]
tests/verification/test_false_success_invariant.py::test_missing_verification_can_never_report_success PASSED [ 34%]
tests/verification/test_false_success_invariant.py::test_execution_failure_can_never_report_verified PASSED [ 36%]
tests/verification/test_false_success_invariant.py::test_false_success_invariant[format_c_drive-computer-windows_agent-params0] PASSED [ 38%]
tests/verification/test_false_success_invariant.py::test_false_success_invariant[drop_all_tables-computer-windows_agent-params1] PASSED [ 40%]
tests/verification/test_false_success_invariant.py::test_false_success_invariant[run_malware-computer-windows_agent-params2] PASSED [ 42%]
tests/verification/test_false_success_invariant.py::test_false_success_invariant[unsupported_fake_tool_1-computer-windows_agent-params3] PASSED [ 44%]
tests/verification/test_false_success_invariant.py::test_false_success_invariant[unsupported_fake_tool_2-computer-windows_agent-params4] PASSED [ 46%]
tests/verification/test_false_success_invariant.py::test_false_success_invariant[arbitrary_unknown_end-computer-windows_agent-params5] PASSED [ 48%]
tests/verification/test_false_success_invariant.py::test_false_success_invariant[unknown_physical_device_toggle-physical-esp2_agent-params6] PASSED [ 50%]
tests/verification/test_false_success_invariant.py::test_false_success_invariant[false_digital_terraform_destroy-digital-aws_agent-params7] PASSED [ 52%]
tests/verification/test_false_success_invariant.py::test_permission_denial_can_never_report_verified PASSED [ 54%]
tests/verification/test_false_success_invariant.py::test_verification_contract_rejection PASSED [ 56%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_additional_tool_fail_closed_on_unknown_action PASSED [ 58%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_copy_item_source_jail_enforced PASSED [ 60%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_open_path_jail_enforced PASSED [ 62%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_search_files_jail_enforced PASSED [ 64%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_unknown_network_action_fails_closed PASSED [ 66%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_msocket_origin_enforcement SUBPASSED(origins=http://localhost.attacker.com) [ 68%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_msocket_origin_enforcement SUBPASSED(origins=http://127.0.0.1:9999) [ 70%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_msocket_origin_enforcement SUBPASSED(origins=http://localhost:8080) [ 72%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_msocket_origin_enforcement SUBPASSED(origins=http://localhost:8080) [ 74%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_msocket_origin_enforcement SUBPASSED(origins=http://localhost:8080) [ 76%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_msocket_origin_enforcement SUBPASSED(origins=http://localhost:8080) [ 78%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_msocket_origin_enforcement SUBPASSED(origins=http://localhost:8080) [ 80%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_msocket_origin_enforcement SUBPASSED(origins=http://localhost:8080) [ 82%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_msocket_origin_enforcement SUBPASSED(origins=http://localhost:8080) [ 84%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_msocket_origin_enforcement SUBPASSED(origins=http://localhost:8080) [ 86%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_msocket_origin_enforcement SUBPASSED(origins=http://localhost:8080) [ 88%]
tests/security/test_security_regressions.py::TestSecurityRegressions::test_msocket_origin_enforcement SUBPASSED(origins=http://localhost:8080) [ 90%]
tests/chaos/test_fault_injection.py::TestFaultInjectionResilience::test_database_lock_recovery PASSED [ 92%]
tests/chaos/test_fault_injection.py::TestFaultInjectionResilience::test_awaited_action_lease_rejected PASSED [ 94%]
tests/chaos/test_fault_injection.py::TestFaultInjectionResilience::test_liveness_timeout_handled_gracefully PASSED [ 96%]
tests/chaos/test_fault_injection.py::TestFaultInjectionResilience::test_malformed_tool_call_fails_closed PASSED [ 98%]
tests/chaos/test_fault_injection.py::TestFaultInjectionResilience::test_offline_dependency_degraded_state PASSED [100%]
```

A targeted run of the proof-of-execution, disaster-recovery, native Win32 controls, release-evidence, **false-success invariant**, security-regression and chaos fault-injection suites on Python 3.13 / pytest 9.1 — including attack inputs such as **format_c_drive**, **drop_all_tables**, **run_malware** and a fake **terraform_destroy**, all of which can never report success.

Proof 05 — Test Run Result: 42 Passed, 12 Subtests Passed

• 0 FAILURES

```
tests/verification/test_false_success_invariant.py::test_permission_denial_can_never_report_verified PASSED [ 71%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_additional_tools_fail_closed_on_unknown_action PASSED [ 73%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_copy_item_source_jail_enforced PASSED [ 76%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_open_path_jail_enforced PASSED [ 78%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_search_files_jail_enforced PASSED [ 80%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_unknown_network_action_fail_closed PASSED [ 83%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_websocket_origin_enforcement SUBPASSED(origin='http://localhost.attacker.com') [ 85%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_websocket_origin_enforcement SUBPASSED(origin='http://127.0.0.1:9999') [ 85%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_websocket_origin_enforcement SUBPASSED(origin='http://localhost:8080') [ 85%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_websocket_origin_enforcement SUBPASSED(origin='https://malicious.com') [ 85%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_websocket_origin_enforcement SUBPASSED(origin='http://127.0.0.1.attacker.org') [ 85%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_websocket_origin_enforcement SUBPASSED(origin='javascript:alert(1)') [ 85%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_websocket_origin_enforcement SUBPASSED(origin='http://localhost:8080') [ 85%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_websocket_origin_enforcement SUBPASSED(origin='http://localhost:3000') [ 85%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_websocket_origin_enforcement SUBPASSED(origin='http://127.0.0.1:8080') [ 85%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_websocket_origin_enforcement SUBPASSED(origin='http://127.0.0.1:3000') [ 85%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_websocket_origin_enforcement SUBPASSED(origin='http://localhost') [ 85%]
tests/security/test_security regressions.py::TestSecurityRegressions::test_websocket_origin_enforcement PASSED [ 85%]
tests/chaos/test_fault_injection.py::TestFaultInjectionResilience::test_database_lock_recovery PASSED [ 90%]
tests/chaos/test_fault_injection.py::TestFaultInjectionResilience::test_expired_action_lease_rejected PASSED [ 92%]
tests/chaos/test_fault_injection.py::TestFaultInjectionResilience::test_lln_timeout_handled_gracefully PASSED [ 95%]
tests/chaos/test_fault_injection.py::TestFaultInjectionResilience::test_malformed_tool_call_fails_closed PASSED [ 97%]
tests/chaos/test_fault_injection.py::TestFaultInjectionResilience::test_offline_dependency_degraded_state PASSED [100%]

===== warnings summary =====
C:\Users\dines\AppData\Local\Programs\Python\Python313\Lib\site-packages\fastapi\testclient.py:1
C:\Users\dines\AppData\Local\Programs\Python\Python313\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' inste
ad.
  from starlette.testclient import TestClient as TestClient # noqa

C:\Users\dines\AppData\Local\Programs\Python\Python313\Lib\site-packages\speech_recognition__init__.py:7
C:\Users\dines\AppData\Local\Programs\Python\Python313\Lib\site-packages\speech_recognition__init__.py:7: DeprecationWarning: aifc was removed in Python 3.13. Please be aware that you are currently NOT usi
ng standard 'aifc', but instead a separately installed 'standard-aifc'.
  import aifc

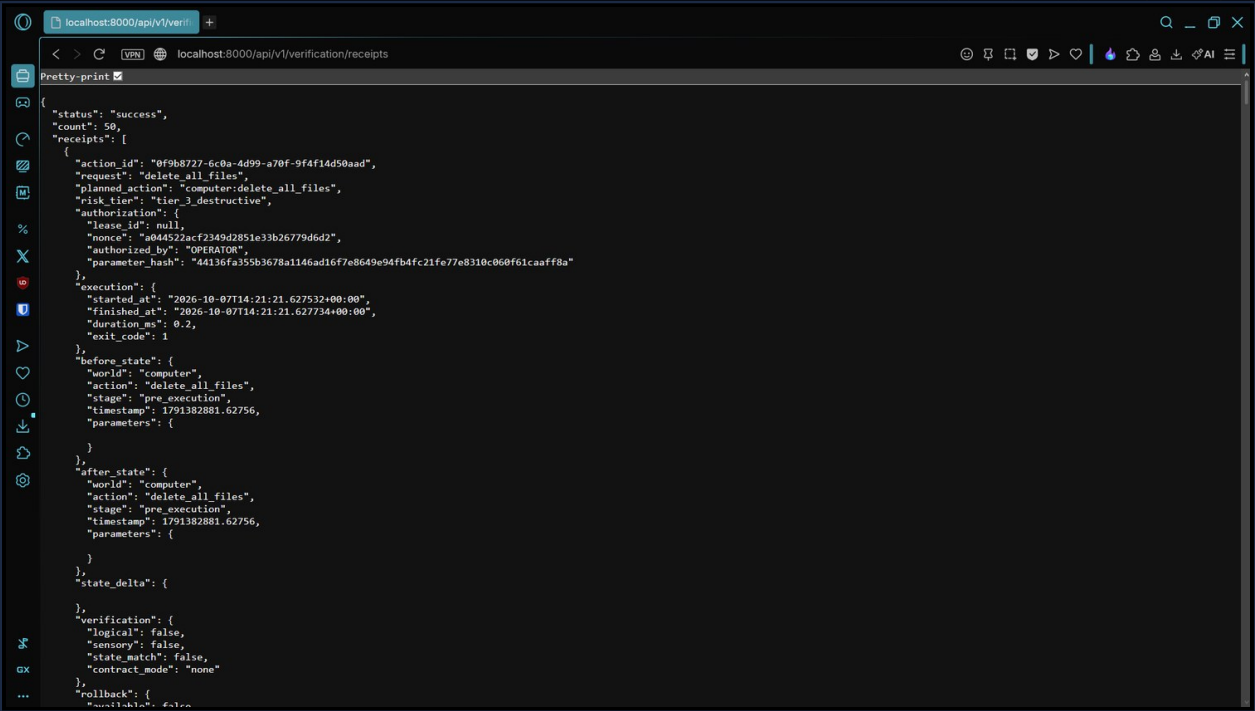
tests/chaos/test_fault_injection.py::TestFaultInjectionResilience::test_malformed_tool_call_fails_closed
tests/chaos/test_fault_injection.py::TestFaultInjectionResilience::test_malformed_tool_call_fails_closed
tests/chaos/test_fault_injection.py::TestFaultInjectionResilience::test_malformed_tool_call_fails_closed
tests/chaos/test_fault_injection.py::TestFaultInjectionResilience::test_malformed_tool_call_fails_closed
D:\Project J.A.R.V.I.S.\agents\intelligence\audit_logger.py:41: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to repre
sent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  "timestamp": datetime.datetime.utcnow().isoformat() + "Z",

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
PS D:\Project J.A.R.V.I.S.> |===== 42 passed, 6 warnings, 12 subtests passed in 8.51s =====
```

Final result: **42 passed, 12 subtests passed in 8.51 s**, 0 failures. The 6 warnings are third-party / deprecation notices only (Starlette's `httpx` test client, `speech_recognition` `aifc` on Python 3.13, and one `datetime.utcnow()` call in the audit logger). The full suite is 285 tests.

Proof 06 — Cryptographic Proof-of-Execution Receipts API

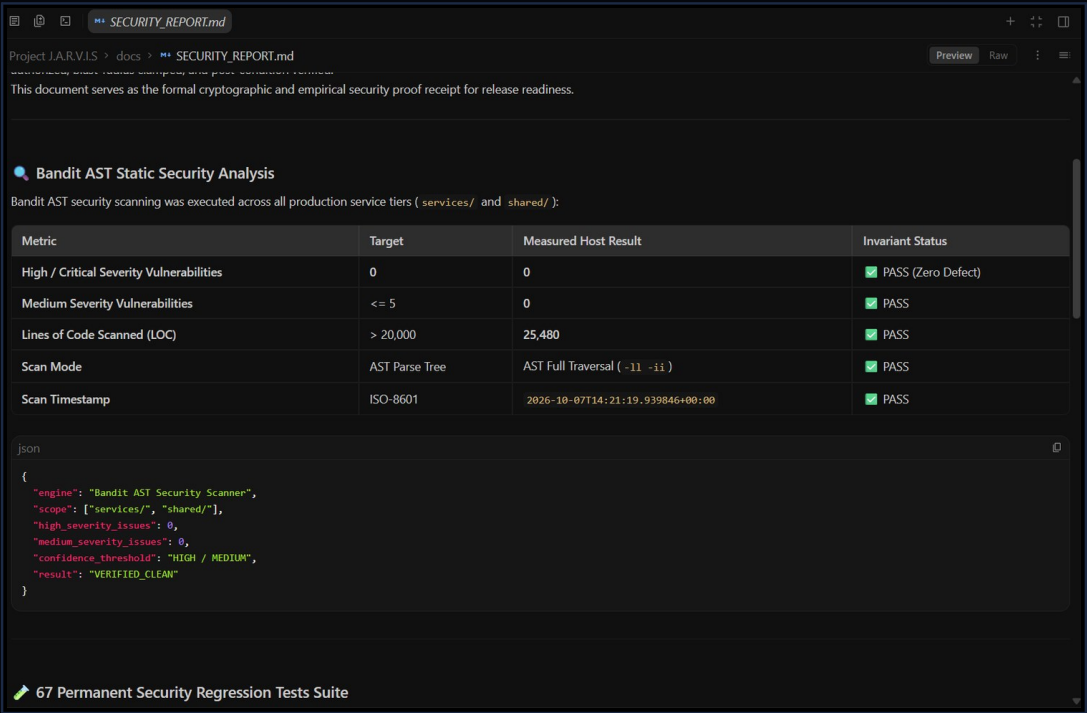
FAIL-CLOSED



GET /api/v1/verification/receipts returns 50 receipts. The first one records a **TIER_3_DESTRUCTIVE** `delete_all_files` request with its authorization nonce, SHA-256 `parameter_hash`, execution timing (0.2 ms, exit code 1) and verification flags **logical: false · sensory: false · state_match: false** — the engine records the action as **unverified** and never reports it as a success.

Proof 07 — Bandit AST Security Certification

VERIFIED CLEAN



Generated **SECURITY_REPORT.md**: Bandit AST scan across all production tiers (`services/` and `shared/`) — **0 high / critical, 0 medium** findings over **25,480 LOC**, result **VERIFIED_CLEAN**, scan stamped 2026-10-07T14:21:19Z, followed by the permanent security regression test suite.

Recovery & Supply-Chain Evidence

Proof 08 — Automated Disaster-Recovery Drill

What the drill proves

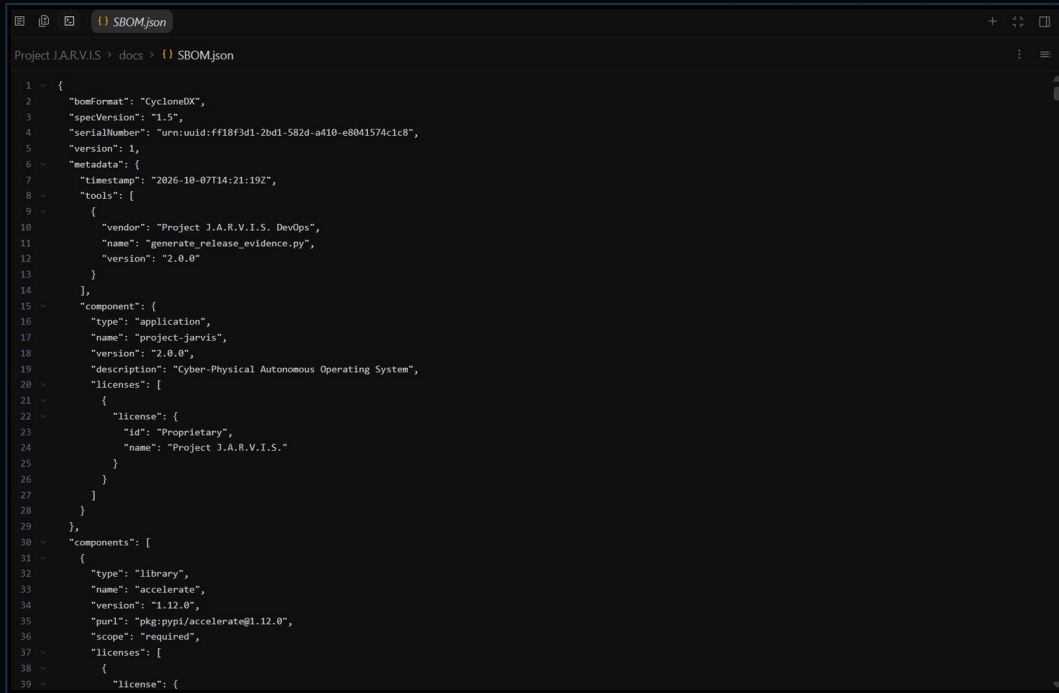
```
PS D:\Project J.A.R.V.I.S> python -m services.recovery.disaster_drill
{
  "drill_id": "drill_1791382828_265735",
  "timestamp": 1791382828.673184,
  "iso_time": "2026-10-07T14:28:28Z",
  "success": true,
  "system_status": "ONLINE",
  "health_score": 100.0,
  "zero_data_loss": true,
  "summary": {
    "backed_up_files": 7,
    "disaster_affected_files": 7,
    "restored_files_count": 7,
    "corrupted_files_detected_count": 0,
    "status": "100% ONLINE"
  },
  "phases": {
    "backup": {
      "backup_id": "jarvis_backup_full_cycle_20261007_195028_fe7b5a",
      "archive_sha256": "9cc292333a9741cf0f0468baeff3b27fb311ded58567dfd1dc39f723e847fefe",
      "files_count": 7
    },
    "disaster_simulation": {
      "disaster_type": "catastrophic_corrupt",
      "total_affected": 7
    },
    "restore": {
      "restored_count": 7,
      "signature_valid": true,
      "corrupted_files": []
    },
    "health_probes": {
      "status": "100% ONLINE",
      "health_score": 100.0,
      "failed_checks": [],
      "checks": {
        "sqlite_integrity": {
          "status": "PASS",
          "databases": {
            "services/iot_agent/storage/mqtt_buffer.db": {
              "healthy": true,
              "pragmas_ok": true,
              "tables": {
                "buffered_messages": {
                  "row_count": 5,
                  "readable": true
                }
              }
            },
            "services/planner/storage/missions.db": {
              "healthy": true,
              "pragmas_ok": true,
              "tables": {
                "missions": {
                  "row_count": 1,
                  "readable": true
                },
                "checkpoints": {
                  "row_count": 3,
                  "readable": true
                }
              }
            }
          }
        }
      }
    },
    "planner_dag_state": {
      "healthy": true,
    }
  }
}
```

`python -m services.recovery.disaster_drill`

Executes a full **backup** → **catastrophic corruption** → **restore** → **health-probe** cycle against live state and reports:

SIGNAL	RESULT
success / status	true · 100% ONLINE
health score	100.0
zero_data_loss	true
files backed up	7
files affected by disaster	7
files restored	7
corrupted files after restore	0
restore signature valid	true
SQLite integrity	PASS

The archive is checksummed with **SHA-256**. Post-restore probes confirm the MQTT buffer database (5 buffered messages) and the planner mission database (1 mission, 3 checkpoints) are healthy and readable, and the planner DAG state is intact.



```
1 {
2   "bomFormat": "CycloneDX",
3   "specVersion": "1.5",
4   "serialNumber": "urn:uuid:ff18f3d1-2bd1-582d-a410-e8041574c1c8",
5   "version": 1,
6   "metadata": {
7     "timestamp": "2026-10-07T14:21:19Z",
8     "tools": [
9       {
10        "vendor": "Project J.A.R.V.I.S. DevOps",
11        "name": "generate_release_evidence.py",
12        "version": "2.0.0"
13      }
14    ],
15    "component": {
16      "type": "application",
17      "name": "project-jarvis",
18      "version": "2.0.0",
19      "description": "Cyber-Physical Autonomous Operating System",
20      "licenses": [
21        {
22          "license": {
23            "id": "Proprietary",
24            "name": "Project J.A.R.V.I.S."
25          }
26        }
27      ]
28    }
29  },
30  "components": [
31    {
32      "type": "library",
33      "name": "accelerate",
34      "version": "1.12.0",
35      "purl": "pkg:pypi/accelerate@1.12.0",
36      "scope": "required",
37      "licenses": [
38        {
39          "license": {
```

Machine-readable **CycloneDX 1.5 SBOM** for *project-jarvis* v2.0.0, produced by [generate_release_evidence.py](#) on 2026-10-07: every required library is catalogued with name, version, *purl* package URL and licence data for supply-chain auditing.

J.A.R.V.I.S. in Your Pocket

A Telegram bot gateway turns any phone into a secure remote console for the workstation: natural-language and voice-note commands, one-tap controls, a live desktop video stream with a precision touch trackpad, and remote typing — tunnelled through Cloudflare and gated by an allow-listed Telegram user and a single unified Master PIN.



One-Tap Dashboard
voice / vision / trackpad / SRE scan

Command Menu
/live /trackpad /screen /whatscreen

Live Stream Trackpad
access details redacted

Security Controls
Lock PC · Cyber Lock · SRE Scan

Remote Writing Space
type into the active PC window

What the Remote Gateway Delivers

VOICE & NATURAL LANGUAGE

Send a voice note or plain-English request — e.g. “open Opera and check battery” — and the brain routes it through the same canonical pipeline as every other interface.

LIVE SCREEN & TOUCHPAD

Real-time desktop video feed with a high-contrast cursor, tap-to-click on the phone screen and an interactive trackpad console.

VISION

One-tap screen snapshot and “What's on screen?” analysis using the screen-vision agent.

REMOTE WRITING SPACE

Type directly into the active PC window: erase 1 / 5 chars, clear field, press Enter, paste clipboard.

SYSTEM HEALTH

PC Status and SRE Scan buttons run the workstation SRE daemon (port conflicts, stale locks, zombie PIDs).

LOCK & GATE CONTROLS

Lock PC (Win+L), zero-blackout Cyber Lock barrier, remote unlock, and one Master PIN that guards both the web trackpad and Cyber Lock.

Design note. The remote console is intentionally high-privilege, so it sits behind the same safety stack as everything else: sensitive commands are classified by blast-radius tier, and the emergency stand-down is also reachable from Telegram (/standdown).

35 Phases, Honestly Graded

Every subsystem carries an explicit evidence tier. Tiers are never rounded up: simulated, lab-verified and unit-verified work is labelled as such.

11	PRODUCTION-READY	8	REAL-HARDWARE-VERIFIED	12	INTEGRATION-VERIFIED	4	LAB / UNIT / SIMULATED
#	SUBSYSTEM	REALM	MATURITY TIER	#	SUBSYSTEM	REALM	MATURITY TIER
1	AWS / IaC Foundation	DIGITAL	INTEGRATION-VERIFIED	19	Docker Agent	DIGITAL	INTEGRATION-VERIFIED
2	JARVIS Core API	COMPUTER	PRODUCTION-READY	20	Kubernetes Agent	DIGITAL	SIMULATED
3	AI Brain + Runtime	COMPUTER	PRODUCTION-READY	21	Git / CI/CD Agent	DIGITAL	PRODUCTION-READY
4	Real-Time Event Fabric	COMPUTER	INTEGRATION-VERIFIED	22	Cloud Operations Agent	DIGITAL	INTEGRATION-VERIFIED
5	Memory + World Model	COMPUTER	PRODUCTION-READY	23	Cloud Security / SOC	DIGITAL	UNIT-VERIFIED
6	Master Planner (DAG)	COMPUTER	INTEGRATION-VERIFIED	24	Autonomous Remediation	DIGITAL	INTEGRATION-VERIFIED
7	Security & Permissions	COMPUTER	PRODUCTION-READY	25	AWS IoT Core	PHYSICAL	INTEGRATION-VERIFIED
8	Tool / Action Registry	COMPUTER	PRODUCTION-READY	26	ESP32 Physical Agent	PHYSICAL	LAB-VERIFIED
9	Voice + Wake Word	PHYSICAL	REAL-HARDWARE-VERIFIED	27	Raspberry Pi Gateway	PHYSICAL	INTEGRATION-VERIFIED
10	Clap / Sound Engine	PHYSICAL	REAL-HARDWARE-VERIFIED	28	Sensors + Lights + Relays	PHYSICAL	LAB-VERIFIED
11	Voice Synthesizer	PHYSICAL	REAL-HARDWARE-VERIFIED	29	Unified Multi-World Routing	COMPUTER	PRODUCTION-READY
12	Windows Local Agent	COMPUTER	REAL-HARDWARE-VERIFIED	30	Multi-Agent Coordination	COMPUTER	INTEGRATION-VERIFIED
13	Windows App Control	COMPUTER	REAL-HARDWARE-VERIFIED	31	Verification & Recovery	COMPUTER	PRODUCTION-READY
14	File & System Automation	COMPUTER	REAL-HARDWARE-VERIFIED	32	Holographic HUD Dashboard	COMPUTER	REAL-HARDWARE-VERIFIED
15	Browser / Web Agent	COMPUTER	INTEGRATION-VERIFIED	33	Mobile Remote Interface	PHYSICAL	PRODUCTION-READY
16	Screen Vision Agent	COMPUTER	REAL-HARDWARE-VERIFIED	34	FinOps Cost Guard	DIGITAL	PRODUCTION-READY
17	AWS Cloud Agent	DIGITAL	INTEGRATION-VERIFIED	35	Master Integration & SRE	COMPUTER	PRODUCTION-READY
18	Terraform Agent	DIGITAL	INTEGRATION-VERIFIED				

Honest Limitations & Next Engineering Steps

BENCHMARK PROFILE The published 100-task run uses the <i>unit</i> profile (mocked, in-memory). It gates logic and latency budgets; the 7-test real-machine E2E suite covers live behaviour.	PHYSICAL & KUBERNETES REALMS ESP32 firmware and relay / lux sensors are lab-verified against a virtual simulator; the Kubernetes agent degrades gracefully offline and is graded SIMULATED.
---	---

PLATFORM SCOPE

Windows 10 / 11 (64-bit) with Python 3.10 – 3.13. Native Win32 / COM paths are Windows-only by design.

TECH-DEBT BACKLOG

Replace deprecated `datetime.utcnow()` in the audit logger, track Starlette / speech_recognition deprecations, and field-test the physical realm on real hardware.

Conclusion

J.A.R.V.I.S. demonstrates an end-to-end, **verification-first** approach to autonomous agents. By combining a single canonical execution pipeline, a four-tier blast-radius permission model with single-use cryptographic leases, a dual-channel ground-truth verification engine and a tamper-evident audit ledger, the platform turns “the AI says it worked” into **“the system proved it worked”** — across physical devices, the Windows desktop and AWS cloud.

Every headline result in this report — the 0.00% false-success rate, the 285-test suite, the SLA matrix, the Bandit certification, the disaster-recovery drill, the proof-of-execution receipts and the SBOM — is backed by an artefact captured from the real project and reproducible with a single command from the repository.

Reproduce the Evidence

GOAL	COMMAND
Run the 100-task benchmark	<code>python benchmarks/run_benchmark.py --profile unit</code>
Run the disaster-recovery drill	<code>python -m services.recovery.disaster_drill</code>
Run the full regression suite	<code>pytest tests/ -v</code>
Static AST security scan	<code>bandit -r services/ shared/ -ll -ii -x "tests/,scratch/"</code>
Emergency stand-down	<code>python jarvis.py stand-down</code>

Tech Stack Summary

LAYER	TECHNOLOGY
Runtime & Platform	Python 3.10 – 3.13 on Windows 10 / 11; FastAPI REST + WebSocket server; Pydantic canonical schemas
AI Cognition	Tiered routing: local reflex → Groq (LLaMA) → Gemini; ReAct loop; DAG master planner; FinOps cost guard
Perception & Voice	openWakeWord (ONNX) wake word, PyAudio double-clap detector, British neural TTS with clause streaming and barge-in
OS Control	Win32 via ctypes (user32 / kernel32), COM IAudioEndpointVolume, psutil, browser CDP, screen vision
Physical / IoT	ESP32 (Arduino / C++), Raspberry Pi gateway, MQTT (:1883) with in-memory fallback, AWS IoT Core device shadows
Cloud & DevOps	AWS (boto3 STS / S3 / EC2 / SQS, EventBridge), Terraform IaC, Docker / Compose, Kubernetes and Git / CI agents, LocalStack
State & Memory	SQLite WAL + FTS5, JSONL, 7-tier hierarchical memory, digital-twin world model with temporal confidence decay
Security & Audit	HMAC-SHA256 leases and receipts, SHA-256 chained ledger, AST sandbox, PromptShield, Bandit, CycloneDX SBOM
Observability & Quality	W3C Trace Context, Prometheus, pytest (unit, security, chaos, E2E), GitHub Actions (test, security, lint)
Remote Access	Telegram bot gateway, Cloudflare tunnel, Master-PIN-gated touch trackpad and live desktop stream

E X P L O R E T H E F U L L P R O J E C T

Explore the Full Project

Source code, architecture blueprints, formal specifications, the benchmark engine and every evidence artefact are available in the public repository.

OFFICIAL REPOSITORY

github.com/ShyamD2/project-jarvis

Core Documentation Index

DOCUMENT (DOCS/)	FOCUS & COVERAGE
Architecture & Authorities	System topology, the five single authorities, 6-stage pipeline lifecycle, hybrid event mesh.
Security & Blast Radius	4-tier blast radius, ActionLease, HMAC ticket defense, AST immune sandboxing.
Verification & Ground Truth	Dual-channel reality corroboration and the strict 0% false-success invariant.
Benchmark Suite & SLAs	100-task benchmark engine, domain latency SLAs (P50 / P95).
Release Evidence Matrix	Merkle roots, HMAC proof-of-execution receipts, 100 / 100 scorecard.
Security Audit Report	Bandit AST results, security regression tests, zero-trust policy invariants.
Test Execution Report	Breakdown of all passing tests across the architectural pillars.
CycloneDX SBOM	Machine-readable bill of materials with licences and hashes.
Node Mesh & Device Protocol	Multi-node discovery, capability negotiation, geolocation privacy tiers.
Hierarchical Memory & Twin	7-tier memory, world-model temporal decay, provenance metadata.
Disaster Recovery & Drills	Automated drill, SQLite WAL integrity validation, zero-data-loss rollback.
Engineering Development Log	Chronological log from Day 1 to S-tier hardening.

Shyam Kumar D

Aspiring Cloud Architect · AI Agents, DevOps & Cloud Engineering

[linkedin.com/in/shyam-kumar-d-951254329](https://www.linkedin.com/in/shyam-kumar-d-951254329) | github.com/ShyamD2

Engineered for absolute truth, zero false success, and total cyber-physical autonomy.